Data Structures and Algorithms - Exam Reference Guide
Table of Contents
Complexity Analysis Tables
Common Time Complexities
Data Structure Operations
Graph Algorithm Complexities
Data Structure Comparisons
Array vs Linked List
Stack vs Queue
BST vs Hash Table
Algorithm Comparisons
Sorting Algorithms
Graph Traversal
Implementation Details
Common Data Structure Operations
Binary Search Tree Operations
Key Formulas and Equations
Complexity Analysis
Graph Theory
Tree Properties
Hash Functions
Quick Reference Guide
Common Algorithm Patterns
Important Properties
Common Pitfalls

Data Structures and Algorithms - Exam Reference Guide

Table of Contents

1. Complexity Analysis Tables
2. Data Structure Comparisons
3. Algorithm Comparisons
4. Implementation Details
5. Key Formulas and Equations

Complexity Analysis Tables

Common Time Complexities

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Binary				

Search	O(1)	O(log n)	O(log n)	O(1)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)
Radix Sort	O(d(n+k))	O(d(n+k))	O(d(n+k))	O(n+k)
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)

Data Structure Operations

Operation	Array	Linked List	Stack	Queue	BST	Hash Table
Access	O(1)	O(n)	O(n)	O(n)	O(log n)	O(1)
Search	O(n)	O(n)	O(n)	O(n)	O(log n)	O(1)
Insert	O(n)	O(1)	O(1)	O(1)	O(log n)	O(1)
Delete	O(n)	O(1)	O(1)	O(1)	O(log n)	O(1)

Graph Algorithm Complexities

Algorithm	Time Complexity	Space Complexity
DFS	O(V + E)	O(V)
BFS	O(V + E)	O(V)
Dijkstra	O(E log V)	O(V)
Bellman-Ford	O(VE)	O(V)
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Kruskal	O(E log E)	O(V)
Prim	$O(V^2)$	O(V)

Data Structure Comparisons

Array vs Linked List

Feature	Array	Linked List
Memory Allocation	Contiguous	Non-contiguous
Size	Fixed	Dynamic
Random Access	O(1)	O(n)
Insert/Delete	O(n)	O(1)
Memory Overhead	None	Extra pointer
Cache Utilization	Better	Poor

Stack vs Queue

Feature	Stack	Queue
Principle	LIFO	FIFO
Operations	push/pop	enqueue/dequeue
Implementation	Array/LL	Array/LL
Applications	Function calls, DFS	BFS, Print queue

BST vs Hash Table

Feature	BST	Hash Table
Order	Maintained	Not maintained
Search	O(log n)	O(1)
Insert	O(log n)	O(1)
Delete	O(log n)	O(1)
Space	O(n)	O(n)

Algorithm Comparisons

Sorting Algorithms

Algorithm	Stable	In-place	Best Case	Worst Case	Space
Bubble	Yes	Yes	O(n)	$O(n^2)$	O(1)
Selection	No	Yes	$O(n^2)$	$O(n^2)$	O(1)
Insertion	Yes	Yes	O(n)	$O(n^2)$	O(1)
Merge	Yes	No	O(n log n)	O(n log n)	O(n)
Quick	No	Yes	O(n log n)	$O(n^2)$	O(log n)
Heap	No	Yes	O(n log n)	O(n log n)	O(1)

Graph Traversal

Feature	DFS	BFS
Data Structure	Stack	Queue
Memory	O(h)	O(w)
Path Finding	Not optimal	Optimal
Cycles	Detects	Detects
Applications	Maze solving, Topological sort	Shortest path, Level order

Implementation Details

Common Data Structure Operations

```
// Stack Operations
push(x) {
  if (top == capacity) throw StackOverflowError();
  arr[++top] = x;
}

pop() {
  if (isEmpty()) throw EmptyStackException();
  return arr[top--];
}

// Queue Operations
enqueue(x) {
  if (isFull()) throw QueueOverflowError();
  rear = (rear + 1) % capacity;
  arr[rear] = x;
}

dequeue() {
  if (isEmpty()) throw EmptyQueueException();
  int x = arr[front];
  front = (front + 1) % capacity;
  return x;
}
```

Binary Search Tree Operations

```
// BST Insert
insert(root, key) {
  if (root == null) return new Node(key);
  if (key < root.key) root.left = insert(root.left, key);
  else if (key > root.key) root.right = insert(root.right, key);
  return root;
}
```

```
// BST Delete
delete(root, key) {
  if (root == null) return root;
  if (key < root.key) root.left = delete(root.left, key);
  else if (key > root.key) root.right = delete(root.right, key);
  else {
    // Node with one or no child
    if (root.left == null) return root.right;
    if (root.right == null) return root.left;
    // Node with two children
    root.key = minValue(root.right);
    root.right = delete(root.right, root.key);
  }
  return root;
}
```

Key Formulas and Equations

Complexity Analysis

- Master Theorem: $T(n) = aT(n/b) + f(n)$
  - If $f(n) = O(n^c)$ where $c < \log_b(a)$: $T(n) = \Theta(n^{\log_b a})$
  - If $f(n) = \Theta(n^c)$ where $c = \log_b(a)$: $T(n) = \Theta(n^c \log n)$
  - If $f(n) = \Omega(n^c)$ where $c > \log_b(a)$: $T(n) = \Theta(f(n))$

Graph Theory

- Euler's Formula: $V - E + F = 2$
- Handshake Lemma: Sum of degrees = $2E$
- Dijkstra's Algorithm: $d[v] = \min(d[v], d[u] + w(u,v))$

Tree Properties

- Height of BST: $\log_2(n) \leq h \leq n-1$
- Number of nodes in complete binary tree: $2^{h+1} - 1$
- Number of leaf nodes in complete binary tree: 2^h

Hash Functions

- Division Method: $h(k) = k \bmod m$
- Multiplication Method: $h(k) = \lfloor m(kA \bmod 1) \rfloor$
- Universal Hashing: $h(k) = ((ak + b) \bmod p) \bmod m$

Quick Reference Guide

Common Algorithm Patterns

1. **Divide and Conquer**
   - Split problem into subproblems
   - Solve subproblems recursively
   - Combine solutions
2. **Dynamic Programming**
   - Break down into overlapping subproblems
   - Store solutions to subproblems
   - Build up solution from stored values
3. **Greedy Approach**
   - Make locally optimal choice
   - Hope for global optimum
   - No backtracking

Important Properties

1. **BST Properties**
   - Left subtree < root < right subtree
   - No duplicate values
   - Inorder traversal gives sorted order
2. **Heap Properties**
   - Complete binary tree
   - Parent > children (max heap)
   - Parent < children (min heap)
3. **Graph Properties**
   - Connected vs Disconnected
   - Directed vs Undirected
   - Weighted vs Unweighted

Common Pitfalls

1. **Time Complexity**
   - Nested loops: multiply complexities
   - Sequential operations: add complexities
   - Recursive calls: use recurrence relations
2. **Space Complexity**
   - Consider recursion stack
   - Account for auxiliary space
   - Watch for memory leaks
3. **Implementation**
   - Handle edge cases
   - Check for null pointers
   - Validate input parameters